

pandas 3.0! 드디어!

성과와 사용성을 중심으로...

한상곤

sigmadream@gmail.com

May 9, 2026

pandas 3.0! 드디어!

pandas 3.0은 문자열(dtype)을 기본값으로 바꾸고 Copy-on-Write를 정식 기본 동작으로 채택해, 예측 가능하고 효율적인 데이터 처리를 지향한 버전입니다. 동시에 연쇄할당(chained assignment) 같은 오래된 관행을 정리하고, `pd.col()` 같은 더 간결한 표현식 문법과 datetime 기본 해상도(resolution) 조정까지 포함해, 2.x에서 누적된 성능 문제를 해결한 버전입니다.

pandas 잘 쓰고 계신가요? 저는 즐겁게 사용하고 있습니다.

저는 pandas로 추천시스템을 설계하고, 책도 쓰고, 학교에서도 연구도 합니다. 여
튼, pandas를 잘 쓰고 있습니다.

소득공제

PDF

eBook

Tidy Data 원칙을 활용한 Pandas 데이터 분석 실무

한상곤 저 | 부크크 | 2026년 02월 20일

첫번째 리뷰어가 되어주세요.

정가

10,000원

웹(Web) 버전으로 보실 수 있습니다.

Copy-on-Write(CoW)

Copy-on-Write(COW)는 “공유하다가 수정이 필요할 때만 복사하는” 최적화 기법입니다. 즉, 처음에는 원본과 복사본이 같은 데이터를 함께 쓰고, 누군가 내용을 바꾸려는 순간에만 새 사본을 만들어 분리하는 방법입니다.

- 변경이 없으면 실제 복사가 일어나지 않음
- 읽기만 하는 경우에는 메모리와 시간이 절약
- 쓰기 시점에만 별도 복사본이 생겨서, 각 객체나 프로세스가 독립적인 상태를 가질 수 있음

장점과 주의점

- 장점

- 불필요한 복사를 줄여서 메모리 사용량과 복사 비용을 낮춤

- 주의점

- 수정이 자주 일어나는 경우에는 결국 복사가 많이 발생해서 이점이 줄어들 수 있음

pandas의 Copy-on-Write(CoW)

- 서브셋이나 메서드 결과는 원본을 변경하지 않는, 복사처럼 동작하는 객체로 간주
- 내부적으로는 필요할 때만 실제 복사하는 **Copy-on-Write**를 사용
- 불필요한 `df = df.copy()` 패턴을 줄일 수 있고, 방어적 복사 감소로 **성능과 메모리 양쪽에 이득**

```
1 # 1. subset 생성 시 메모리 복사가 발생하지 않음 (뷰 공유)
2 subset = df.iloc[0:2]
3 # 2. 데이터를 수정하는 순간 실제 복사(Copy-on-Write) 발생
4 # 원본 df는 변경되지 않으며, SettingWithCopyWarning도 없음
5 subset.iloc[0, 1] = "pandas 3.0"
```

일관성

- 파이썬 슬라이스와 Copy-on-Write(CoW)는 비슷해 보이지만, 세부적인 사항에서 차이가 발생
 - 파이썬 슬라이스(slice)는 보통 새 시퀀스를 만들어 반환하는 복사 계열 동작
 - CoW는 겉으로는 복사본처럼 보이지만 실제 복사는 수정 시점까지 미루는 방식
- 최종 사용자 입장에서 결론적으로 파이썬 사용자의 경험과 일치하는 사용 경험을 제공
 - pandas 2.x는 파이썬 사용자의 경험과 일치하지 않는 부분이 존재
 - pandas 3.x는 기존의 파이썬 사용자의 경험과 일치하도록 변경

파이썬의 슬라이스

```
1 l = [1, 2, 3, 4, 5]
2 l1 = l[0:2]
3 print(l)    # [1,2,3,4,5]
4 print(l1)   # [1,2]
5
6 l1[0] = 100
7 print(l)    # [1,2,3,4,5]
8 print(l1)   # [100,2]
```


pandas 2.x의 Copy-on-Write(CoW)

```
1 df = pd.DataFrame({'a':[1,2], 'b':[3.14, 3.15]})
2 df1 = df.iloc[0:1]
3
4 df1.iloc[0, 0] = 100
5 print(df)
6 #      a      b
7 # 0  100  3.14
8 # 1   2  3.15
```

pandas 3.x의 Copy-on-Write(CoW)

```
1 df = pd.DataFrame({"a": [1, 2], "b": [3.14, 3.15]})
2 df1 = df.iloc[0:1]
3 df1.iloc[0, 0] = 100
4 print(df1) # 100  3.14
5 print(df)
6 #      a      b
7 # 0    1  3.14
8 # 1    2  3.15
```

문자열 **str** 기본 사용

pandas 3.0에서 문자열 컬럼은 기본적으로 NumPy **object** dtype이 아니라 새로운 **str** dtype으로 추론됩니다. PyArrow가 설치되어 있으면 이 문자열 dtype이 내부적으로 Arrow 기반으로 동작해서 성능과 메모리 효율이 좋아집니다.

문자열 dtype 변화

예전에는 `pd.Series(["a", "b"])`가 보통 `object` dtype이었지만, pandas 3.0에서는 기본적으로 `str` dtype으로 추론됩니다.

```
1 df = pd.DataFrame({"a": [1, 2, 3], "b": ["hello", "world", "!"]})
2 print(df.dtypes) # a: int64, b: object -> a: int64, b: str
```

- `str` dtype은 문자열 전용이라서 type safety가 더 좋고, 문자열 컬럼 처리에 더 적합
 - `string[pyarrow]` 타입도 확인 가능
- `if col.dtype == object`로 문자열을 체크하던 코드가 의도대로 동작하지 않을 수 있기 때문에, `pd.api.types.is_string_dtype()` 사용을 권장

PyArrow

PyArrow가 설치되어 있으면 Arrow 기반으로 추론하여 **메모리 효율이 개선**

Apache Arrow는 “열 단위로 데이터를 빠르고 효율적으로 저장하고 주고받기 위한 표준 메모리 형식과 생태계”라고 이해하면 좋습니다.

- Arrow는 단순한 라이브러리라기보다, 여러 언어와 시스템이 공통으로 사용할 수 있는 데이터 표현 규약에 가깝음 → pandas, Polars, DuckDB, Parquet, Spark 계열 도구들과 연결될 때 강점이 큼
- PyArrow는 그 Apache Arrow를 파이썬에서 쓸 수 있게 만든 라이브러리

왜 좋은가?

- Arrow는 Python object 보다 메모리 구조가 단순하고 연속적이라 캐시 친화적
- 문자열 같은 컬럼에서도 개별 Python 문자열 객체를 많이 만드는 방식보다 메모리 오버헤드를 줄일 수 있음
- 멀티스레드 처리와 벡터화된 연산에 잘 맞는 구조라서, 파일 읽기나 변환 단계에서 성능 이점
- pandas 3.0의 문자열 dtype이나 Copy-on-Write 같은 변화와도 방향이 잘 맞음

언제 쓰면 좋은가?

PyArrow는 “대용량 분석과 데이터 이동”에 큰 장점을 가집니다.

- 추천
 - 문자열 컬럼이 많고 메모리 사용량이 큰 데이터셋
 - CSV나 Parquet를 자주 읽고 쓰는 ETL 작업
 - pandas와 다른 분석 엔진 사이의 데이터 교환
 - 컬럼 일부만 자주 읽는 분석 파이프라인
- 비추천
 - 아주 작은 데이터나 단순 스크립트

UDF 가속을 위한 외부 엔진 인터페이스

pandas 3에서 UDF(user defined function)를 위한 외부 엔진 지원, 즉 **engine** 매개변수를 통해 UDF를 외부 실행 엔진에 맡기는 새로운 실행 인터페이스를 제공합니다.

- 복잡해서 벡터화하기 어려운 UDF를 외부 엔진(bodo.ai JIT)으로 실행할 수 있도록 **engine 인터페이스**를 추가
- 벡터화 버전보다도 더 빠른 경우가 보고되고 있고, 데이터가 커질수록 상대적 이득이 커지는 방향

기존의 UDF 문제

pandas에서는 `df.apply(func, axis=1)`나 `Series.map` 같은 방식으로 사용자 정의 함수(UDF)를 써서, 열별, 행별로 커스텀 로직을 실행할 수 있었지만, 내부적으로는 Python 스칼라 함수를 한 행씩 호출하는 식으로 동작해 느렸습니다.

`add_people` 같은 스칼라 UDF를 쓰면, 벡터화된 버전보다 수천 배 느려질 수 있습니다. 반대로, 완전히 벡터화된 코드를 쓰면 빠르지만 복잡해져서 가독성이 떨어집니다.

```
1 def add_people(row):  
2     return row["max_people"] + row["max_children"]  
3 rooms.apply(add_people, axis=1)
```

외부 실행 엔진

pandas 3에서는 **apply**, **map** 같은 UDF 호출에 **engine** 인자를 추가해, UDF를 독립된 실행 엔진이 JIT 컴파일하거나 벡터화해서 돌리도록 확장했습니다.

```
1 import bodo
2 df.apply(format_room_info, axis=1, engine=bodo.jit())
```

- `bodo.jit()`가 engine으로 들어가면, pandas가 아니라 bodo.ai 엔진이 이 UDF를 JIT 컴파일해서 실행, 성능에 관한 내용은 공식 [벤치마크](#)를 참고
- `blosc2`(NumPy 스타일 작업을 압축된 메모리로 가속) 등 다양한 종류의 계산, 최적화, 병렬 분산에 대한 지원을 기대해 볼 수 있음

정리

- 문자열 타입 변경
- Copy-on-Write
- Arrow 및 외부 실행 엔진
- 기타

문자열 dtype 기본값 변경에 따른 동작 차이

기존 코드를 급격하게 3.0으로 옮기기 보다는 2.3 환경에서 미리 문자열 타입 관련 경고를 해결하는 것이 좋음

- `Series(["a", "b"])`의 dtype이 더 이상 `object`가 아니라 `str`로 표시되며, 문자열 관련 메서드 구현과 누락값 처리 등에서 세부 동작이 달라질 수 있음
- 2.3에서 제공된 “문자열 dtype 관련 경고”를 처리하지 않은 코드는 3.0에서 실제 동작 변경이나 에러를 겪을 수 있음

Copy-on-Write로 인한 체인 할당 금지

기존 체인 할당 코드들은 반드시 수정

- `df["foo"][df["bar"] > 5] = 100` 같은 체인 할당은 이제 경고가 아니라 오류(`ChainedAssignmentError`)를 내며, 반드시 `df.loc[df["bar"] > 5, "foo"] = 100` 같이 한 단계 할당으로 작성
- 이로 인해 `SettingWithCopyWarning`을 없애려고 넣던 무분별한 `.copy()` 호출이 필요 없어지고, 코드 의미가 더 명확해지는 대신 기존 체인 할당 코드들은 반드시 수정

Arrow 및 외부 엔진과의 연계 강화

- 문자열 `str` dtype의 Arrow 백엔드, **Arrow PyCapsule**, UDF용 **engine** 파라미터 등은 pandas를 Arrow, JIT 엔진 등과 긴밀히 엮어 대규모 데이터와 생산 환경에서의 성능을 개선하는 방향성을 제시
- 실무에서는 대규모 문자열 처리나 복잡한 UDF가 있는 파이프라인에서 Arrow 백엔드와 외부 엔진을 조합해 점진적으로 튜닝하는 전략을 취할 수 있음

pd.col() 도입으로 assign 표현식 간소화

- DataFrame.assign에서 람다 대신 `pd.col("a") + pd.col("b")`처럼 컬럼 기반 표현식을 직접 쓸 수 있음
- 정적 분석이나 타입 검사 측면에서도 더 다루기 쉬운 형태로 발전하는 방향

```
1 df = pd.DataFrame({'a': [1, 1, 2], 'b': [4, 5, 6]})
2 df.assign(c = lambda df: df['a'] + df['b'])
3 df.assign(c = pd.col('a') + pd.col('b')) # <- 3.x에서 지원
```

datetime 기본 해상도 변경

극단적인 날짜 범위를 다루는 코드에서는 기존 결과와의 미세한 차이를 확인할 필요 있음

- datetime, timedelta의 기본 해상도가 나노초에서 마이크로초 또는 입력 해상도로 바뀌어, 1678년 이전이나 2262년 이후 날짜에서 발생하던 범위 초과 문제를 줄임

릴리스의 전반적 방향성

- 커뮤니티 평을 보면 눈에 띄는 “새 기능”보다는 문자열 dtype, Copy-on-Write, pd.col, Arrow 연계 같은 근본적인 토대를 다지는 릴리스로 보는 **시각**
- 새 코드베이스라면 애초부터 3.0의 동작을 전제로 작성하고, 기존 프로젝트는 문자열 처리, 체인 할당, datetime 범위, deprecated API 사용 여부 네 가지를 우선적으로 점검하는 것이 효율적

당부

바로 3.0으로 올리기보다 먼저 2.3으로 업그레이드하고, 경고를 모두 해결한 뒤 3.0으로 가는 것이 공식적으로 권장됨

- 오래된 파라미터, 비권장 경로, 호환성 유지용 별칭 등 deprecated 상태였던 많은 기능이 실제로 제거되었으며, 전체 목록은 3.0 릴리스 노트의 **backwards incompatible API changes** 섹션에 정리
- 초기 3.0.x 버전의 경우 CoW 오류 등을 수정해 나가고 있으니, 3.x 사용자들은 가 능하면 최신 버전으로 업데이트를 주기적으로 유지하는 것을 권장

Q&A

질문 있으신가요?